

Dynamisch Docker-omgevingbeheer voor CTF-events op on-prem infrastructuur.

Dynamisch Docker-omgevingbeheer voor CTF-events.

Manu Devloo.

Scriptie voorgedragen tot het bekomen van de graad van
Professionele bachelor in de toegepaste informatica

Promotor: Dhr. T. Clauwaert

Co-promotor: Dhr. L. Singier

Academiejaar: 2025–2026

Eerste examenperiode

Departement IT en Digitale Innovatie .

**HO
GENT**

Woord vooraf

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Samenvatting

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Inhoudsopgave

Lijst van figuren	vi
--------------------------	-----------

Lijst van tabellen	vii
---------------------------	------------

Lijst van codefragmenten	viii
---------------------------------	-------------

1 Inleiding	1
1.1 Probleemstelling	1
1.2 Onderzoeksvraag	2
1.3 Onderzoeksdoelstelling	2
1.4 Scope en afbakening	2
1.5 Opzet van deze bachelorproef	3
2 Achtergrond en begrippenkader	4
2.1 Capture The Flag (CTF)	4
2.1.1 Types challenges	4
2.1.2 Deployment van challenges: statisch vs. dynamisch	5
2.1.3 Bekende CTF-platformen	5
2.2 CTFd als platform	5
2.2.1 Architectuur van CTFd	5
2.2.2 Het plugin-systeem	5
2.2.3 Challenge-beheer en granulariteit	5
2.3 Docker en containerisatie	6
2.3.1 Wat is Docker?	6
2.3.2 Docker Engine API	6
2.3.3 Docker SDK voor Python	6
2.3.4 Docker Compose	6
2.3.5 Resource management	6
2.3.6 Netwerkisolatie	7
2.4 Beveiliging van containers	7
2.4.1 Beveiligingsrisico's bij dynamisch aanmaken van containers	7
2.4.2 Low-hanging fruit: basismaatregelen	7
2.4.3 Container escape	8
2.4.4 Resource exhaustion en DoS-risico's	8

2.5	Load balancing zonder Kubernetes	8
2.5.1	Waarom geen Kubernetes?	8
2.5.2	Alternatieven	8
2.5.3	Wanneer is schaling nodig?	8
3	Methodologie	9
4	Huidige situatie	10
4.1	Organisatie van CTF-events op eigen infrastructuur	10
4.2	Manuele container deployments	10
4.3	Overzicht van bestaande tools en hun tekortkomingen	10
4.4	Conclusie: wat ontbreekt er?	10
5	Gewenste situatie	11
5.1	Vereisten voor de oplossing	11
5.1.1	Functionele vereisten	11
5.1.2	Niet-functionele vereisten	11
5.2	Architectuurvisie	11
5.3	Granulariteitsbeslissing: container per gebruiker of per team	11
5.4	Schaalbaarheidsvereisten	11
6	Proof-of-Concept	12
6.1	Architectuur van de PoC	12
6.2	Implementatie van de CTFd-plugin	12
6.2.1	Technische keuzes	12
6.2.2	Container lifecycle management	12
6.2.3	Configuratie van een challenge-container	12
6.2.4	Gebruikersinterface in CTFd	12
6.3	Beveiliging in de PoC	12
6.3.1	Isolatie tussen containers van verschillende teams/gebruikers	12
6.4	Schaalbaarheid en load balancing	12
6.5	Automatisatie en gebruiksgemak voor organisatoren	12
7	Validatie en testing	13
7.1	Testplan	13
7.1.1	Doelstellingen	13
7.1.2	Testomgeving	13
7.2	Functionele tests	13
7.2.1	Aanmaken en beëindigen van containers	13
7.2.2	Isolatie tussen gebruikers	13
7.3	Beveiligingstests	13
7.3.1	Container escape pogingen	13
7.3.2	Resource exhaustion simulaties	13

7.4	Load testing	13
7.4.1	Testopzet	13
7.4.2	Meetpunten	13
7.4.3	Resultaten en conclusies.	13
8	Conclusie	14
8.1	Beantwoording van de onderzoeksvragen	14
8.2	Beperkingen van de PoC	14
8.3	Aanbevelingen voor productiegebruik	14
8.4	Mogelijke uitbreidingen.	14
A	Onderzoeksvoorstel	15
A.1	Introductie	15
A.1.1	Probleemstelling	15
A.1.2	Onderzoeksvraag.	15
A.1.3	Onderzoeksdoelstelling	16
A.2	State-of-the-art	16
A.3	Methodologie	17
A.4	Verwacht resultaat, conclusie	17
B	Installatie- en beheerinstructies	19
C	Configuratiebestanden	20
D	Testresultaten in detail	21
E	Onderzoekslog	22

Lijst van figuren

Lijst van tabellen

Lijst van codefragmenten

1

Inleiding

Capture The Flag (CTF)-events zijn cybersecuritywedstrijden waarbij deelnemers technische uitdagingen oplossen om punten te scoren (**Innovirtuoso2025**). Ze worden georganiseerd door hogescholen, universiteiten, bedrijven en onafhankelijke gemeenschappen als educatieve en competitieve oefeningen. Elke deelnemer of elk team werkt aan zogenaamde *challenges*: geïsoleerde omgevingen met een kwetsbaarheid die uitgebuit moet worden om een *flag* te bemachtigen.

Naarmate CTF-events populairder worden, groeit ook de behoefte aan robuuste, schaalbare infrastructuur om deze challenges te hosten. Vandaag de dag worden challenges op eigen infrastructuur dikwijls manueel of semi-automatisch beheerd, wat leidt tot operationele overhead, beveiligingsrisico's en schaalbaarheidsknelpunten (**Singier2025**).

1.1. Probleemstelling

Bij het organiseren van CTF-events op eigen (on-premise) infrastructuur is het dynamisch en veilig beheren van containers voor challenges een bijzonder complexe opgave. Organisatoren moeten voor elke challenge een geïsoleerde containeromgeving opzetten, beheren en weer afbreken op basis van gebruikersacties. Dit proces verloopt vandaag grotendeels manueel of met eenvoudige scripts, wat de volgende problemen met zich meebrengt:

- Manuele interventie bij het opstarten en stopzetten van containers kost tijd en is foutgevoelig.
- Er is geen gegarandeerde isolatie tussen containers van verschillende teams of gebruikers.
- Resource-uitputting (CPU, RAM) door ongecontroleerde containers vormt een risico voor de stabiliteit van de host.

- Bestaande cloud-gebaseerde platforms (bijv. hosted CTFd) zijn niet altijd geschikt voor organisaties die werken met privédata of op eigen hardware willen draaien.

De doelgroep van dit onderzoek zijn CTF-organisatoren en systeembeheerders die events hosten op eigen on-premise infrastructuur en op zoek zijn naar een geautomatiseerde, veilige en schaalbare oplossing.

1.2. Onderzoeksvraag

De centrale onderzoeksvraag luidt:

Hoe kunnen Docker-containers die gelinkt zijn aan CTFd-challenges dynamisch worden beheerd, aangemaakt en beëindigd op basis van gebruikersacties en ingestelde parameters, op on-premise infrastructuur?

Hieruit volgen de volgende deelvragen:

- Welke beveiligingsmaatregelen zijn minimaal vereist om container-escapes en misbruik te voorkomen?
- Op welke manier kunnen resource-limieten (CPU, RAM) en netwerkisolatie technisch worden afgedwongen?
- Hoe kan een CTFd-plugin de Docker Engine API aanspreken zonder zware externe afhankelijkheden?
- Hoe valideert men de stabiliteit en veiligheid van de oplossing onder zware belasting?

1.3. Onderzoeksdoelstelling

Het beoogde resultaat van deze bachelorproef is een werkende proof-of-concept (PoC) die aantoont hoe CTFd-challenges dynamisch en veilig beheerd kunnen worden op eigen infrastructuur. De PoC omvat een CTFd-plugin die communiceert met de Docker Engine API, resource-limieten afdwingt en netwerkisolatie garandeert. De oplossing is volledig self-hosted, open-source en modulair uitbreidbaar.

1.4. Scope en afbakening

Dit onderzoek beperkt zich tot on-premise, self-hosted omgevingen. Kubernetes wordt bewust niet als primaire oplossing gekozen omwille van de operationele complexiteit die niet in verhouding staat met de schaal van een typisch CTF-event. Cloudplatforms en betaalde diensten vallen buiten scope. De focus ligt op open-source tooling die draait op standaard Linux-servers.

1.5. Opzet van deze bachelorproef

De rest van deze bachelorproef is als volgt opgebouwd:

In Hoofdstuk 2 wordt de achtergrond en het begrippenkader geschetst: wat zijn CTF-events, hoe werkt CTfD, wat zijn de relevante Docker-concepten en welke beveiligingsaspecten spelen een rol?

In Hoofdstuk 3 wordt de onderzoeksmethodologie toegelicht.

In Hoofdstuk 4 wordt de huidige situatie (AS-IS) geanalyseerd: hoe worden CTF-events vandaag beheerd en wat zijn de tekortkomingen?

In Hoofdstuk 5 wordt de gewenste situatie (TO-BE) beschreven, inclusief de architectuurvisie en de vereisten voor de oplossing.

In Hoofdstuk 6 wordt de proof-of-concept uitgewerkt: architectuur, implementatie, beveiliging en schaalbaarheid.

In Hoofdstuk 7 worden de testresultaten besproken: functionele tests, beveiligingstests en load tests.

In Hoofdstuk 8, tenslotte, wordt de conclusie gegeven en een antwoord geformuleerd op de onderzoeksvragen, samen met aanbevelingen voor toekomstig werk.

2

Achtergrond en begrippenkader

Dit hoofdstuk biedt de lezer alle achtergrondkennis die nodig is om de rest van de bachelorproef te begrijpen. Iemand die niet vertrouwd is met CTF-events, Docker of containerbeveiliging, beschikt na het lezen van dit hoofdstuk over een solide basis. Achtereenvolgens komen CTF-events, het CTFd-platform, Docker en containerisatie, containerbeveiliging en load balancing zonder Kubernetes aan bod.

2.1. Capture The Flag (CTF)

Een Capture The Flag (CTF)-event is een cybersecuritywedstrijd waarbij deelnemers technische uitdagingen moeten oplossen om een geheime reeks tekens — de *flag* — te bemachtigen en in te dienen voor punten (**Innovirtuoso2025**). CTF-events worden ingezet als educatief middel, als rekruteringsstool en als competitieve sport binnen de cybersecuritygemeenschap.

2.1.1. Types challenges

CTF-challenges zijn doorgaans opgedeeld in categorieën:

- **Web:** kwetsbaarheden in webapplicaties zoals SQL-injectie, Cross-Site Scripting (XSS) en onveilige autorisatie.
- **Pwn (Binary Exploitation):** het exploiteren van kwetsbaarheden in gecompileerde binaire bestanden, bijv. buffer overflows.
- **Reverse Engineering:** analyse en begrip van gecompileerde software zonder broncode.
- **Cryptografie:** het kraken of omzeilen van cryptografische systemen en protocollen.

- **Forensics:** het reconstrueren van gebeurtenissen uit digitale artefacten zoals logbestanden, geheugenafbeeldingen of netwerkopnamen.

2.1.2. Deployment van challenges: statisch vs. dynamisch

Challenges kunnen op twee manieren worden uitgerold. Bij een **statische deployment** wordt één gedeelde instantie van de challenge opgezet die alle deelnemers tegelijk gebruiken. Dit is eenvoudig te beheren, maar brengt risico's mee: één deelnemer kan de omgeving voor alle anderen beïnvloeden. Bij een **dynamische deployment** krijgt elk team of elke gebruiker een eigen, geïsoleerde containerinstantie. Dit verhoogt de integriteit van de wedstrijd maar vereist automatisering (**CTFd**).

2.1.3. Bekende CTF-platformen

De meest gebruikte platformen voor het organiseren van CTF-events zijn:

- **CTFd:** het meest wijdverspreide open-source platform, met ondersteuning voor plugins (**CTFdGitHub**).
- **rCTF:** een alternatief platform ontwikkeld door het redpwn-team (**rCTF**).
- **kCTF:** een door Google ontwikkeld Kubernetes-gebaseerd platform, ontworpen voor grote-schaal events (**kCTF**).

2.2. CTFd als platform

CTFd is een open-source CTF-platform gebouwd in Python met het Flask-framework (**CTFdGitHub**). Het biedt een gebruiksvriendelijke interface voor zowel organisatoren als deelnemers en is modulair uitbreidbaar via een plugin-systeem.

2.2.1. Architectuur van CTFd

CTFd bestaat uit een Flask-backend, een relationele database (standaard SQLite, in productie MySQL of PostgreSQL) en een frontend op basis van Bootstrap. De applicatie draait typisch achter een reverse proxy zoals Nginx. Alle functionaliteit is bereikbaar via een REST API.

2.2.2. Het plugin-systeem

CTFd ondersteunt plugins die extra functionaliteit toevoegen aan het platform. Een plugin is een Python-pakket dat in de CTFd/plugins/-map geplaatst wordt. Plugins kunnen nieuwe challenge-types registreren, routes toevoegen en bestaande logica uitbreiden (**CTFdGitHub**).

2.2.3. Challenge-beheer en granulariteit

CTFd ondersteunt standaard statische challenges. Voor dynamische instanties — waarbij elke gebruiker of elk team een eigen containerinstantie krijgt — zijn plugins

vereist, zoals de CTFd-Whale plugin (**CTFdWhale**). De granulariteit (container per gebruiker of per team) is een ontwerpbeslissing met impact op resource-gebruik en isolatie.

2.3. Docker en containerisatie

2.3.1. Wat is Docker?

Docker is een platform voor het bouwen, distribueren en draaien van applicaties in containers (**Arxiv2024**). Een container is een geïsoleerd, lichtgewicht proces dat gebruikmaakt van Linux-kernelmechanismen (namespaces, cgroups) om te worden afgeschermd van andere processen en van de host.

Containers versus virtuele machines: waar een virtuele machine een volledig besturingssysteem emuleert, deelt een container de kernel van de host. Dit maakt containers veel sneller op te starten en minder resource-intensief, maar vergt zorgvuldigere beveiligingsconfiguratie.

2.3.2. Docker Engine API

De Docker Engine API is een RESTful HTTP-API die het mogelijk maakt om programmatisch te communiceren met de Docker-daemon (**DockerEngineAPI**). Via deze API kunnen containers worden aangemaakt, gestart, gestopt en verwijderd, en kunnen netwerken en volumes worden beheerd. De API is de basis voor alle Docker-tooling, inclusief de Docker CLI.

2.3.3. Docker SDK voor Python

De Docker SDK voor Python is een officiële client-bibliotheek die de Docker Engine API omhult en in Python aanroept (**DockerSDKPython**). Het is de aanbevolen manier om Docker programmatisch te besturen vanuit Python-applicaties, zoals een CTFd-plugin.

2.3.4. Docker Compose

Docker Compose is een tool voor het definiëren en draaien van multi-container applicaties via een YAML-configuratiebestand (**DockerCompose**). Voor challenge-configuraties biedt Compose een declaratieve manier om containerparameters (image, netwerk, resource-limieten) te definiëren.

2.3.5. Resource management

Docker biedt mechanismen voor het beperken van resource-gebruik per container, gebaseerd op Linux cgroups:

- CPU-limieten: `-cpus` en `-cpu-shares`
- Geheugenlimieten: `-memory` en `-memory-swap`

- I/O-limieten via `blkio-cgroup-parameters`

2.3.6. Netwerkisolatie

Docker biedt verschillende netwerkmodi:

- **Bridge:** de standaardmodus waarbij containers communiceren via een virtuele brug, maar gescheiden zijn van de host.
- **None:** geen netwerktoegang.
- **Custom overlay/bridge networks:** containers kunnen worden ingedeeld in geïsoleerde virtuele netwerken.

2.4. Beveiliging van containers

2.4.1. Beveiligingsrisico's bij dynamisch aanmaken van containers

Wanneer containers dynamisch worden aangemaakt op basis van gebruikersinvoer, ontstaan extra risico's: een kwaadaardige deelnemer kan proberen de configuratie te manipuleren om bevoorrechte toegang te krijgen of om resources te monopoliseren.

2.4.2. Low-hanging fruit: basismaatregelen

De OWASP Docker Security Cheat Sheet (**OWASP2024**) en artikelen over containerbeveiliging (**Aikido2025**) identificeren een reeks basismaatregelen:

- **Privilege management:** containers draaien standaard als root; dit moet worden voorkomen via `-user`.
- **Read-only filesystems:** via `-read-only` worden schrijfacties beperkt.
- **Capabilities droppen:** Linux capabilities die niet nodig zijn (bijv. `NET_ADMIN`, `SYS_ADMIN`) worden verwijderd via `-cap-drop ALL`.
- **Seccomp-profielen:** beperken de systeemoproepen die een container mag uitvoeren.
- **AppArmor/SELinux:** MAC-profielen die extra toegangscontrole bieden.
- **No new privileges:** het `-security-opt no-new-privileges`-vlag voorkomt privilege-escalatie.

Het CIS Docker Benchmark (**CISDocker**) biedt een uitgebreide checklist van aanbevolen beveiligingsinstellingen voor Docker-hosts en -containers.

2.4.3. Container escape

Een container escape is een aanval waarbij een kwaadaardige actor de isolatielaag van de container doorbreekt en toegang verkrijgt tot de host of andere containers. Bekende technieken maken gebruik van kwetsbaarheden in de Docker-daemon, misbruik van gemounte sockets (`/var/run/docker.sock`) of misconfiguraties (bijv. privileged mode) (NVD).

2.4.4. Resource exhaustion en DoS-risico's

Zonder resource-limieten kan een deelnemer de hostserver overbelasten door een excessief aantal containers te starten of door een container alle CPU- en geheugen-capaciteit te laten consumeren. Dit vormt een *Denial-of-Service* (DoS)-risico voor alle overige deelnemers.

2.5. Load balancing zonder Kubernetes

2.5.1. Waarom geen Kubernetes?

Kubernetes is het de-facto standaardplatform voor containerorkestratie op schaal (**kCTF**). Het biedt ingebouwde load balancing, automatische schaling en zelfherstel. Voor kleinschalige CTF-events wegen de voordelen echter niet op tegen de operationele complexiteit: Kubernetes vereist een substantieel cluster, diepgaande kennis en aanzienlijke overhead voor beheer.

2.5.2. Alternatieven

- **Docker Swarm:** Docker's ingebouwde orkestratielaag voor meerdere nodes (**DockerSwarm**). Eenvoudiger dan Kubernetes, maar minder uitgebreid.
- **Traefik:** een dynamische reverse proxy die automatisch routes configureert op basis van Docker-labels (**Traefik**). Ideaal als *entry point* voor dynamisch aangemaakte containers.
- **HAProxy:** een betrouwbare, performante TCP/HTTP load balancer (**HAProxy**) voor geavanceerdere routeringsconfiguraties.
- **Nginx:** een veelgebruikte webserver die ook als reverse proxy en load balancer kan fungeren.

2.5.3. Wanneer is schaling nodig?

Bij CTF-events varieert de belasting sterk: er zijn piekbelastingen bij de start van een event wanneer veel deelnemers tegelijk challenges starten. De infrastructuur moet deze piekbelasting kunnen opvangen zonder de stabiliteit van het platform te compromitteren.

3

Methodologie

Dit onderzoek combineert een literatuurstudie met praktijkgericht werk. Het verloopt in vier opeenvolgende fasen.

Fase 1 – Literatuurstudie en begrippenkader (Hoofdstuk 2): Via literatuurstudie worden de relevante concepten en de state-of-the-art in kaart gebracht. De focus ligt op CTFd, de Docker Engine API, containerbeveiliging en load balancing op on-premise infrastructuur.

Fase 2 – Analyse van de huidige en gewenste situatie (Hoofdstukken 4 en 5): De huidige aanpak voor het beheren van CTF-challenges op eigen infrastructuur wordt geanalyseerd (AS-IS). Op basis hiervan en de literatuurstudie worden functionele en niet-functionele vereisten opgesteld en een architectuurvisie uitgetekend (TO-BE).

Fase 3 – Proof-of-Concept (Hoofdstuk 6): Er wordt een werkende PoC ontwikkeld: een CTFd-plugin die via de Docker Engine API containers dynamisch beheert. De PoC implementeert resource-limieten, netwerkisolatie en de basisbeveiligingsmaatregelen die in de literatuurstudie zijn geïdentificeerd. De schaalbaarheid wordt getest met een multi-VM opzet en een reverse proxy.

Fase 4 – Validatie en testing (Hoofdstuk 7): De PoC wordt gevalideerd aan de hand van een testplan met functionele tests, beveiligingstests (container escape pogingen, resource exhaustion) en load tests met tools zoals k6 (**k6**) of Locust (**Locust**).

4

Huidige situatie

Dit hoofdstuk analyseert hoe CTF-events vandaag worden georganiseerd op eigen infrastructuur. Het brengt de problemen en beperkingen van de huidige aanpak in kaart als vertrekpunt voor de gewenste oplossing.

4.1. Organisatie van CTF-events op eigen infrastructuur

4.2. Manuele container deployments

4.3. Overzicht van bestaande tools en hun tekortkomingen

4.4. Conclusie: wat ontbreekt er?

5

Gewenste situatie

Op basis van de tekortkomingen die in het vorige hoofdstuk werden geïdentificeerd, beschrijft dit hoofdstuk de vereisten voor de oplossing en de architectuurvisie voor het gewenste systeem.

5.1. Vereisten voor de oplossing

5.1.1. Functionele vereisten

5.1.2. Niet-functionele vereisten

5.2. Architectuurvisie

5.3. Granulariteitsbeslissing: container per gebruiker of per team

5.4. Schaalbaarheidsvereisten

6

Proof-of-Concept

Dit hoofdstuk beschrijft de uitwerking van de proof-of-concept (PoC). Na de architectuurbespreking komen de implementatiedetails van de CTFd-plugin, de beveiligingsmaatregelen, de schaalbaarheidsopzet en het gebruiksgemak voor organisatoren aan bod.

6.1. Architectuur van de PoC

6.2. Implementatie van de CTFd-plugin

6.2.1. Technische keuzes

6.2.2. Container lifecycle management

6.2.3. Configuratie van een challenge-container

6.2.4. Gebruikersinterface in CTFd

6.3. Beveiliging in de PoC

6.3.1. Isolatie tussen containers van verschillende teams/gebruikers

6.4. Schaalbaarheid en load balancing

6.5. Automatisatie en gebruiksgemak voor organisatoren

7

Validatie en testing

Dit hoofdstuk beschrijft het testplan en de testresultaten waarmee de correctheid, veiligheid en schaalbaarheid van de PoC worden aangetoond.

7.1. Testplan

7.1.1. Doelstellingen

7.1.2. Testomgeving

7.2. Functionele tests

7.2.1. Aanmaken en beëindigen van containers

7.2.2. Isolatie tussen gebruikers

7.3. Beveiligingstests

7.3.1. Container escape pogingen

7.3.2. Resource exhaustion simulaties

7.4. Load testing

7.4.1. Testopzet

7.4.2. Meetpunten

7.4.3. Resultaten en conclusies

8

Conclusie

- 8.1. Beantwoording van de onderzoeksvragen**
- 8.2. Beperkingen van de PoC**
- 8.3. Aanbevelingen voor productiegebruik**
- 8.4. Mogelijke uitbreidingen**



Onderzoeksvoorstel

Het onderwerp van deze bachelorproef is gebaseerd op een onderzoeksvoorstel dat vooraf werd beoordeeld door de promotor. Dat voorstel is opgenomen in deze bijlage.

A.1. Introductie

Dit onderzoeksvoorstel richt zich op het dynamisch beheer van Docker-omgevingen tijdens Capture The Flag (CTF) events (**Innovirtuoso2025**) op on-premise infrastructuur. Het idee voor dit onderzoek komt voort uit de noodzaak om efficiënt en veilig challenges te beheren (**Singier2025**).

A.1.1. Probleemstelling

De centrale probleemstelling luidt: Bestaat er een manier om dynamisch Docker-omgevingbeheer te doen op on-prem infrastructuur tijdens Capture The Flag events? Het beheren van containers voor challenges vereist een robuust systeem dat kan omgaan met gebruikersacties en variabele belastingen, zonder in te boeten op veiligheid of performance.

Daarnaast is het een vereiste dat de oplossing volledig self-hosted kan worden ingezet. Hiervoor zijn drie belangrijke redenen: flexibiliteit (mogelijkheid tot uitbreidingen en features), kostprijs (het betalen van on-premise infrastructuur versus cloud-diensten is voor veel organisaties een andere financiële flow) en de voorkeur voor open-source oplossingen.

A.1.2. Onderzoeksvraag

De hoofdonderzoeksvraag van dit bachelorproefvoorstel is:

Hoe kunnen we dynamisch onze docker containers gelinkt aan CTFd-challenges beheren, aanmaken en beëindigen op basis van gebruikers-

acties en ingestelde variabelen?

Om deze hoofdvraag te beantwoorden, worden de volgende deelvragen geformuleerd, opgesplitst in probleem- en oplossingsdomein:

Probleemdomein:

- Welke fundamentele en direct toepasbare beveiligingsmaatregelen ('low-hanging fruit') zijn noodzakelijk om de veiligheid van de Docker-host en -containers te waarborgen?
- Welke specifieke beveiligingsrisico's brengt het dynamisch aanmaken van containers door gebruikers met zich mee op on-premise infrastructuur?
- Wat is de impact van frequente container-creatie en -vernietiging op de performance van de host-server?
- Welke communicatieprotocollen zijn geschikt voor de interactie tussen het CTF-platform en de container-runtime?

Oplossingsdomein (gericht op de Proof-of-Concept):

- Hoe kan een CTFd-plugin ontwikkeld worden die rechtstreeks communiceert met de Docker Engine API zonder zware externe dependencies?
- Op welke manier kunnen resource-limieten (CPU, RAM) en netwerkisolatie technisch worden afgedwongen in de PoC?
- Hoe kan de architectuur zo worden opgezet dat deze volledig self-hosted en modulair uitbreidbaar is?
- Hoe valideren we de stabiliteit van de oplossing onder gesimuleerde zware belasting (load testing)?

A.1.3. Onderzoeksdoelstelling

Het doel is om een werkende oplossing of proof-of-concept te realiseren die aan toont hoe CTFd-challenges dynamisch en veilig beheerd kunnen worden op eigen infrastructuur.

A.2. State-of-the-art

Er is al documentatie beschikbaar over het deployen van challenges binnen CTFd (**CTFd**). Echter, het veilig en dynamisch beheren van deze containers op eigen infrastructuur brengt specifieke uitdagingen met zich mee.

Beveiliging is een cruciaal aspect; containers moeten geïsoleerd zijn om te voorkomen dat deelnemers dingen kunnen doen op de host of andere containers. Bronnen zoals de Docker Security Cheat Sheet van OWASP (**OWASP2024**) en artikelen over container security (**Aikido2025**) bieden richtlijnen hiervoor.

Daarnaast biedt een overzicht van het Docker-ecosysteem inzicht in de beschikbare tools en technologieën die ingezet kunnen worden (**Arxiv2024**). Het onderzoek zal zich baseren op deze bronnen om een veilige en schaalbare architectuur te ontwerpen.

A.3. Methodologie

Het onderzoek zal bestaan uit een combinatie van literatuurstudie en praktijkgericht onderzoek.

In de eerste fase wordt via literatuurstudie onderzocht welke 'low-hanging fruit' beveiligingsmaatregelen essentieel zijn. De focus ligt hierbij op praktische en effectieve best practices voor container security (zoals resource limits, netwerkisolatie en privilege management) in plaats van diepgaande theoretische beveiligingsmodellen. Daarnaast wordt gekeken naar de Docker Engine API en SDKs voor de integratie.

Vervolgens zal een proof-of-concept (PoC) worden ontwikkeld waarin een CTFd-omgeving wordt opgezet die communiceert met een Docker-daemon. Hierbij wordt gefocust op de implementatie van een API-interface, het instellen van resource limieten en het toepassen van de geïdentificeerde basisbeveiliging.

Tot slot zal de oplossing gevalideerd worden aan de hand van een uitgebreid testplan als onderdeel van de PoC-verificatie. Dit testplan beschrijft de strategie om zowel de stabiliteit als de veiligheid van het systeem te waarborgen. Concreet worden er load tests uitgevoerd om de schaalbaarheid en performance onder zware belasting te meten. Daarnaast worden specifieke beveiligingstests (zoals container escape pogingen en resource exhaustion simulaties) uitgevoerd om te verifiëren of de geïmplementeerde isolatie en restricties effectief zijn tegen misbruik.

A.4. Verwacht resultaat, conclusie

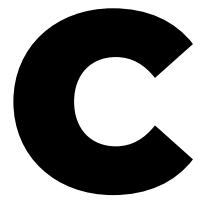
Het verwachte resultaat is een gedocumenteerde en werkende implementatie voor dynamisch containerbeheer binnen CTFd. Dit omvat:

- Een adviesrapport over de te gebruiken architectuur en beveiligingsmaatregelen.
- Een proof-of-concept implementatie.
- Testresultaten die de schaalbaarheid en stabiliteit aantonen.
- Installatie- en beheerinstructies om de reproduceerbaarheid van de PoC te waarborgen.
- Technische documentatie van de ontwikkelde architectuur, zodat derden hierop kunnen verder bouwen.

De meerwaarde van dit onderzoek ligt in het vereenvoudigen van het beheer van CTF-events op eigen hardware, met behoud van veiligheid en performance.

B

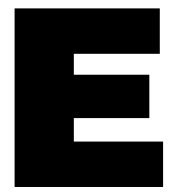
Installatie- en beheerinstructies



Configuratiebestanden

D

Testresultaten in detail



Onderzoekslog